

Operations	Big-O	Notes
Remove	$O(n)$	Removal would require shifting all the subsequent elements to the left by one & that takes $O(n)$.
Remove (at the end)	$O(1)$	Special case of remove where no other element need to be shifted.

Thing to look out for during interviews

- clarify if there are duplicates value in the array. Would the presence of duplicate value affect the answer? Does it make the question simpler or harder?

$\text{if}(i > 0 \ \&\& \text{arr}[i] == \text{arr}[i-1]) \text{continue}; \rightarrow \text{skips Dups}$

- When using an index to iterate through array element, be careful not to go out bound.

general rule for calculating Range $\rightarrow$

Your Goal	Start from	Why?
you need to compare $\text{arr}[i]$ with $\text{arr}[i+1]$	$i = n-2$	Because $i+1$ must be $\leq n-1$
you need to compare $\text{arr}[i]$ with $\text{arr}[i-1]$	$i = 1$	because $i-1$ must be ≥ 0
Normal Scan	$i = 0$	first element is included
reverse normal Scan	$i = n-1$	start at last element

Common Access Pattern

Access Pattern $\leadsto [0 \leq \text{idx} < \text{arr.size}()]$

$\text{arr}[i] \leadsto$ Required Range for $i [0 \leq i < n]$

$\text{arr}[i+1] \leadsto 0 \leq i < n-1$

$\text{arr}[i-1] \leadsto 1 \leq i < n$

$\text{arr}[i+k] \leadsto 0 \leq i < n-k-1$

$\text{arr}[i-k] \leadsto k \leq i < n$

$\text{arr}[\text{left}], \text{arr}[\text{right}] \leadsto 0 \leq \text{left} \leq \text{right} < n$

$\text{arr}[\text{mid}-1], \text{arr}[\text{mid}+1] \leadsto 1 \leq \text{mid} \leq n-2$

Safe Start/End

$\leadsto$ any

$\leadsto$ start at $n-2$ if going backward

$\leadsto$ start at 1 if going forward

$\leadsto i \leq n-k-1$

$\leadsto i \geq k$

$\leadsto \text{left} = 0, \text{right} = n-1$

$\text{mid} = (\text{low} + \text{high}) / 2$

where $\text{low} \geq 1$ & $\text{high} \leq n-2$

- Be mindful about slicing or concatenating array in your code. Typically slicing and concatenation array would take $O(n)$ time.
- Use start and end indices to demarcate a subarray/range where possible

Corner Cases

- Empty sequence
- Sequence with 1 or 2 element
- Sequence with repeated Element
- Duplicates values in the sequence.

Techniques

↳ Note that because both array and strings are sequence (a string is an array of characters), most of techniques here will apply to string problems.

1) Sliding Window

↳ This applies to many subarray/substring problems.

In a sliding window, the two pointers usually move in the same direction will never overtake each other. This ensure that each value is only visited at most twice and the time complexity is still $O(n)$.

2) Two pointers

↳ This pointer is a more general version of sliding window where the pointer can cross each other and can be on different arrays.

When you are given two array to process, it's common to have one index per array (pointer) to traverse/compare the both of them, incrementing one of the pointers when relevant.

3) Traversing from the right.

Sometimes you can traverse the array starting from the right instead of the conventional approach of the left.

4) Sorting the Array

Is the array sorted or partially sorted? if it is, some form of binary search be possible. This also usually mean that the interviewer is looking for a solution that is faster than $O(n)$.

can you sort the array? Sometimes sorting the array first may significantly simplify the problem. Obviously this would not work if the order of array element need to be preserved.

5) Precomputation

For questions where summation or multiplication of a subarray is involved, pre-computation using hashing or a prefix/suffix sum/product might be useful.

6) Index as a hash key

if you are given a sequence and the interviewer asks for a $O(1)$ space, it might be possible to use the array itself as a hash table. for example $\rightarrow$ if the array only has value from 1 to N , where N is the length of the array, negate the value at that index (minus one) to indicate presence of that number.

7) Traversing the array more than Once

This might be obvious, but traversing the array twice/thrice (as long as fewer than n times) is still $O(n)$. Sometimes traversing the array more than once can help you solve the problem while keeping the time complexity to $O(n)$.